

Deployment & CI / CD

How CI tests + builds images on every push, how CD auto-deploys from a green main, the required secrets, and the manual / rollback operations.

End-to-end guide: from a blank VDS to a live, self-deploying production stack. Once it's set up, every `git push origin main` ends with the new version rolling on the server about fifteen minutes later — no manual SSH.

What this covers

1. The end-to-end flow at a glance
2. The accounts you need (provider list + price ceiling)
3. **First-time VDS setup** — the one-hour, one-time walkthrough
4. The **CI** pipeline that runs on every push
5. The **CD** pipeline that auto-deploys from a green `main`
6. Choosing the mail FROM address
7. Required GitHub repo secrets
8. Manual operations + rollback
9. Failure handling

End-to-end flow

```
git push origin main
  |
  v
+-----+
|   GitHub CI   | ci.yml - ~12-15 min
| test · lint · | build + push images to GHCR
| sonar · build |
+-----+
|   CI green    |
  v
+-----+
|   GitHub CD   | deploy.yml - ~2-3 min
| SSH → VDS     | git pull · compose pull · up -d
+-----+
  |
  v
VDS rolling forward
(pre-built GHCR images)
```

End-to-end: ~17 minutes from `git push` to the new version live. After the initial setup, no manual SSH is ever needed.

Prerequisites — accounts to create

SERVICE	WHAT FOR	RECOMMENDED	COST
Domain + DNS + TLS	one place for everything	Cloudflare (registrar + DNS + free proxy TLS)	~\$10/year
VDS (compute)	running the stack	Contabo Cloud VPS (16 GB+ x86, EU) or Hetzner	~\$9–25/month
Mail relay (SMTP)	outbound mail (verify, 2FA, alerts)	Brevo (300/day free ≈ 9k/month) or Resend	\$0 for low volume
GitHub	code + CI + image registry	GitHub (GHCR comes with the repo)	\$0

Total typical cost: ~\$10/year + \$9–25/month.

First-time VDS setup

A one-time, ~1-hour walkthrough done from scratch on a fresh server. After this the CD pipeline handles everything.

1. Provision the VDS

- **Spec:** ≥16 GB RAM (the stack runs OpenSearch + Kafka + Keycloak + two JVMs + Postgres), x86/amd64, Ubuntu 24.04 LTS. ~100 GB NVMe is plenty for code + Postgres + indices + image cache. EU region for low latency from TR.
- Order from your chosen provider (Contabo Cloud VPS 20 NVMe and similar tiers fit the bill).
- During order, paste your laptop's SSH public key (next step) into the provider's "SSH public key" field — most providers offer this and skip generating a root password.

 VDS provisioning screencast — TODO *Record*: Contabo Cloud panel → New VPS → choose plan + region + Ubuntu 24.04 + paste SSH pub key → order *Save as:* `docs/imagesandgifs/setup-vds-provision.gif`

2. SSH access from your laptop

Generate a key once on your machine (no passphrase — required for unattended deploys):

```
ssh-keygen -t ed25519 -C "your-comment" -f ~/.ssh/id_ed25519 -N ""
cat ~/.ssh/id_ed25519.pub          # paste this into the provider
```

After the VDS is provisioned, verify the key login works:

```
ssh root@<VDS_IP> 'uname -a'
```

If the provider auto-generated a password instead of accepting the key, install it once with that password:

```
ssh-copy-id -i ~/.ssh/id_ed25519.pub root@<VDS_IP>
```

3. Initial hardening — disable password authentication

Once key login works, drop the password door entirely. Bots constantly brute-force SSH on the open internet — turning passwords off closes that attack surface.

On the server, in `/etc/ssh/sshd_config`, set `PasswordAuthentication no`. Ubuntu cloud images usually re-enable it inside `/etc/ssh/sshd_config.d/50-cloud-init.conf` — change it there too. Then `systemctl reload sshd`. Verify with `sshd -T | grep -i passwordauth`.

Your key login keeps working; anything else is rejected. **Don't lose the private key on your laptop** — back it up to a password manager or Time Machine.

4. Install Docker + Compose plugin

The official one-shot script (Ubuntu / Debian / RHEL):

```
ssh root@<VDS_IP> 'curl -fsSL https://get.docker.com | sh'
ssh root@<VDS_IP> 'docker --version && docker compose version'
```

5. GitHub deploy key — let the VDS clone the private repo

The repo is private; the VDS isn't a "user" with a GitHub login. Solution: a per-server SSH key registered as a **read-only Deploy Key** on the repo.

On the VDS, generate a dedicated key (`ssh-keygen -t ed25519 -f /root/.ssh/github`), tell SSH to use it for `github.com` in `/root/.ssh/config`:

```
Host github.com
  IdentityFile /root/.ssh/github
```

then copy the public key (`/root/.ssh/github.pub`) into the next step.

In GitHub:

1. *Settings* → *Deploy keys* → *Add deploy key*
2. Title: `VDS - <provider>`
3. Paste the public key
4. **Leave "Allow write access" unchecked** — read-only. If the key ever leaks, attackers can clone (the source becomes public eventually anyway) but cannot push or destroy.
5. Add key

6. Clone the repo on the VDS

```
ssh root@<VDS_IP>
cd /root
git clone git@github.com:<owner>/<repo>.git
cd <repo>
```

7. Cloudflare DNS + free HTTPS

Cloudflare gives DNS + TLS termination for free if your domain is on its DNS.

In the Cloudflare dashboard → `<your-domain>` → **DNS** → **Records** → **Add record**:

TYPE	NAME	VALUE	PROXY
A	@	<VDS_IP>	Proxied (orange cloud)
A	www	<VDS_IP>	Proxied

The orange-cloud proxy gives you free HTTPS at the edge (no Let's Encrypt to manage), DDoS protection, and a CDN. Your VDS keeps serving plain HTTP on port 80; Cloudflare terminates TLS.

 Cloudflare DNS screencast — TODO *Record*: Cloudflare → your-domain → DNS → Records → Add record (A @ + A www, both Proxied) Save as: docs/imagesandgifs/setup-cloudflare-dns.gif

8. Mail relay — Brevo example

Brevo's free tier (300/day ≈ 9,000/month) is plenty for transactional mail (account verify, 2FA, alerts). Same flow for Resend / SES with minor differences.

1. Sign up at `app.brevo.com` (no card needed for free).
2. **Senders & Domains** → **Add domain** → enter `your-domain.com`.
3. Brevo gives you 2-3 DNS records to verify (DKIM CNAMEs). If your DNS is on Cloudflare, Brevo can install them automatically with one click ("Authorize") — much easier than pasting them by hand.
4. Wait for verification to flip green (1-10 minutes).
5. **Settings** → **SMTP & API** → **Generate a new SMTP key**. Copy the long key once — Brevo won't show it again.

You'll plug `smtp-relay.brevo.com:587` + your Brevo SMTP login + that key into the `.env` in the next step.

 Brevo SMTP screencast — TODO *Record*: Brevo → Senders & Domains → Add domain → Authorize Cloudflare → Settings → SMTP & API → Generate SMTP key Save as: docs/imagesandgifs/setup-brevo-smtp.gif

9. Production `.env`

The repo ships a `.env.example` with every variable documented. Copy and fill in:

```
cd /root/<repo>
cp .env.example .env
chmod 600 .env
nano .env
```

Replace the placeholders with strong random values from your password manager (or `pwgen`).

OpenSearch needs 8+ characters with upper, lower, digit, and a special character — respect that or the container won't start.

Critical sections:

- **Login:** `FINANCE_ADMIN_PASSWORD`, `DEMO_USER_PASSWORD` — the demo-visible accounts (keep simple if it's a showcase; randomise for a real install).
- **Infra:** `KEYCLOAK_ADMIN_PASSWORD`, `DB_PASSWORD`, `REDIS_PASSWORD`, `LDAP_ADMIN_PASSWORD`, `LDAP_CONFIG_PASSWORD`, `OPENSEARCH_PASSWORD` (OpenSearch needs ≥ 8 chars with upper + lower + digit + special — the inline command above produces a compliant one).
- **Mail:** `SPRING_MAIL_HOST=smtp-relay.brevo.com`, `SPRING_MAIL_PORT=587`, `SPRING_MAIL_USERNAME=<brevo SMTP login>`, `SPRING_MAIL_PASSWORD=<brevo SMTP key>`, `SPRING_MAIL_PROPERTIES_MAIL_SMTP_AUTH=true`, `SPRING_MAIL_PROPERTIES_MAIL_SMTP_STARTTLS_ENABLE=true`, `NOTIFICATION_FROM_ADDRESS=noreply@your-domain.com`
- **Domain / frontend:** `FRONTEND_BASE_URL=https://your-domain.com`, `VITE_KEYCLOAK_URL=https://your-domain.com/auth`. These are baked into the frontend at build time — set them before the first build.

Demo / showcase tip: with `make demo` or the `docker-compose.demo.yml` overlay you get a seeded portfolio and a `demouser` account; in that mode you don't need any market-data API keys (`EVDS_API_KEY` / `COINGECKO_KEY` can be empty). Set `APP_MARKET_INIT_ENABLED=false` so the backend doesn't try a cold-start fetch.

10. First deploy

The first deploy builds the images on the server (CI hasn't pushed any yet). Use the demo overlay if you want seeded data; layer the prod overlay on top to skip the Mailpit dev catcher and send mail through Brevo:

```
docker compose -f docker-compose.yml -f docker-compose.demo.yml -f docker-compose.prod.yml up
-d --build
```

First build is ~25–35 minutes (Maven downloads + JVM compile + Vite + Docker layers + db-seed loading the 46 MB market snapshot). Watch it with `docker compose logs -f`.

11. Verify it's up

```
docker compose ps                # everything Up + healthy
curl -fsS https://your-domain.com/actuator/health  # should be 200
curl -fsS https://your-domain.com/                # frontend HTML
```

Open `https://your-domain.com` in a browser. Login with the admin credentials from `.env`. Trigger a real action (price alert / 2FA / account verify) and watch the email show up at your real inbox (it goes out through Brevo with proper DKIM, not Mailpit).

That's the one-time setup done. From here on, every `git push origin main` deploys automatically.

CI pipeline (`.github/workflows/ci.yml`)

Triggered on every pull request and every push to `main`.

JOB	WHAT IT DOES	DEPENDS ON
<code>finance-common-test</code>	builds + tests the shared library, uploads JaCoCo XML	—
<code>backend-test</code>	installs finance-common, runs the backend Maven reactor (<code>./mvnw verify</code>), uploads per-module JaCoCo XML	<code>finance-common-test</code>
<code>notification-test</code>	builds + tests the notification microservice, uploads JaCoCo XML	<code>finance-common-test</code>
<code>frontend-lint</code>	<code>npm ci && npm run lint</code>	—
<code>sonar</code>	SonarQube scan, downloads JaCoCo artifacts (gated on <code>vars.SONAR_HOST_URL</code>)	all the above
<code>docker-backend</code>	builds + pushes the backend image to GHCR	<code>backend-test</code> , <code>finance-common-test</code>
<code>docker-notification-backend</code>	same for notification	<code>notification-test</code> , <code>finance-common-test</code>
<code>docker-frontend</code>	same for frontend (with <code>VITE_*</code> build args from repo variables)	<code>frontend-lint</code>
<code>compose-validate</code>	sanity-checks the merged compose resolves cleanly	all docker-*

Tests + lint run on **every PR**. Image builds and the push to GHCR only run on **push to `main`**, so PRs never pollute the registry. JaCoCo XML is uploaded as a workflow artifact so the Sonar job can read it across modules.

CD pipeline (`.github/workflows/deploy.yml`)

Triggered automatically when CI succeeds on `main`, or manually via the Actions UI:

```
on:
  workflow_dispatch:
  workflow_run:
    workflows: ["CI"]
    types: [completed]
    branches: [main]

jobs:
  deploy:
    if: github.event_name == 'workflow_dispatch'
    || github.event.workflow_run.conclusion == 'success'
```

Steps (over SSH to the VDS):

1. `git pull --ff-only` — compose files + mounted config (small, seconds)
2. `docker login ghcr.io` with the workflow's auto-issued `GITHUB_TOKEN` (read-only on the repo's own packages, so private images can be pulled with zero stored PATs)
3. `docker compose pull` — fetches the new images from GHCR
4. `docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d` — rolling restart of only the services whose image hash changed
5. `docker image prune -f` — cleans up the old image versions

No source compile on the server, no Maven, no npm. Pull + restart only. ~2-3 minutes end-to-end.

Mail — choosing the FROM address

The mail provider (Brevo / Resend / SES) is the SMTP relay; it isn't a mailbox. You don't need to "create an account" for `noreply@your-domain.com` — once the domain is verified in the provider, you can send from **any address on that domain** and the provider signs it with DKIM so receivers accept it as legitimate.

Common conventions:

ADDRESS	USE
<code>noreply@your-domain.com</code>	Signals "don't reply, this is automated". Most common.
<code>hi@your-domain.com</code>	Friendlier, e.g. for product newsletters.
<code>admin@your-domain.com</code>	System / ops alerts.

Set the chosen address in `NOTIFICATION_FROM_ADDRESS` in `.env`. To **receive** replies you need actual email hosting (Google Workspace, Zoho, Fastmail ...) — that's a separate concern from sending and isn't required for the app to work.

Image registry

CI builds three custom images and pushes them to **GitHub Container Registry (GHCR)**, tagged with both `latest` and the short commit SHA:

```
ghcr.io/<owner>/finance-portal-backend
ghcr.io/<owner>/finance-portal-notification-backend
ghcr.io/<owner>/finance-portal-frontend
```

Private registry by default (matches the repo's visibility). The VDS pulls with a fine-grained PAT scoped to `read:packages`. All third-party images (Postgres, Kafka, Keycloak, OpenSearch, Redis, Nginx, ...) come from their public official registries unchanged.

Required GitHub repo secrets

Under **Settings** → **Secrets and variables** → **Actions**:

SECRET	PURPOSE
<code>VDS_HOST</code>	Server IP or hostname
<code>VDS_USER</code>	SSH user on the server
<code>VDS_SSH_KEY</code>	Private key matching the server's <code>~/.ssh/authorized_keys</code>
<code>VDS_APP_DIR</code>	Absolute path of the cloned repo on the server (e.g. <code>/root/<repo></code>)

The deploy workflow pulls images using the auto-issued `GITHUB_TOKEN`, so no extra PAT is needed for GHCR access.

 GitHub Secrets screencast — *TODO Record*: Repo → Settings → Secrets and variables → Actions → New repository secret (add all four VDS_* entries) *Save as*: `docs/imagesandgifs/setup-github-secrets.gif`

Optional, only with self-hosted Sonar:

- `SONAR_TOKEN` (secret) and `SONAR_HOST_URL` (variable)

Manual operations

Force a deploy from any commit without touching code:

```
gh workflow run deploy.yml
```

Or: Actions → *Deploy to VDS* → **Run workflow**.

Pin a specific image version (rollback to a known-good SHA): on the server, set

```
BACKEND_IMAGE=ghcr.io/.../finance-portal-backend:<sha> in .env and run docker compose -f
```


`docker-compose.yml -f docker-compose.prod.yml up -d backend`. The compose file reads `${BACKEND_IMAGE:-...}`, so any tag can be pinned through env without editing source.

Check what's running:

```
ssh root@vds 'cd app && docker compose ps'
ssh root@vds 'cd app && docker compose logs -f --tail=200 backend'
```

Stop the stack temporarily (to save VDS cost):

```
ssh root@vds 'cd app && docker compose down' # keeps data
```

Failure handling

- A failing test, lint, or Sonar quality gate **fails CI** and stops the pipeline — no image is pushed, no deploy fires. The running version on the VDS is untouched.
- A failing image build (`docker-*`) likewise blocks deploy. CI must be green for `deploy.yml` to fire.
- If `deploy.yml` itself fails (SSH timeout, registry login error, compose error), the previous version keeps running on the VDS — `docker compose pull` only pulls; the running container is replaced only when `up -d` succeeds.
- A container failing to start after `up -d` is visible in `docker compose ps` and the container's logs (`docker compose logs <service>`). Roll back to the previous SHA with the image-tag override above.

Why this shape

- CI tests + image build are split into separate jobs so the slow ones (Maven reactor, Docker buildx) run in parallel with the fast ones (lint, finance-common). Failures surface faster.
- CD is a separate workflow (`workflow_run`) instead of one more job in CI, so deploys honour CI's exact pass/fail and never race the image push.
- Images are pre-built in CI rather than on the server: the VDS doesn't need Maven, npm, build tools or large heap — only Docker. Deploys are fast, the server stays small, and every environment runs the exact same image bytes that CI tested.
- Cloudflare in front of the VDS gives free HTTPS, free DDoS protection, and a CDN cache for the static frontend — without any TLS certificate to manage on the server.
- One single SMTP block in `.env` drives **both** the app's transactional mail and Keycloak's account emails (verify / 2FA / reset), so switching mail provider is one config block to update, not two.

Local vs VDS — what's actually different

Same compose, same code, same images. Only a few `.env` keys change:

KEY	LOCAL DEFAULT	VDS VALUE
FRONTEND_BASE_URL	http://localhost	https://your-domain.com
VITE_API_BASE_URL	/api/v1 (same-origin)	/api/v1 (same-origin, unchanged)
VITE_KEYCLOAK_URL	http://localhost/auth	https://your-domain.com/auth
SPRING_MAIL_HOST	mailpit (catcher)	smtp-relay.brevo.com
SPRING_MAIL_PORT	1025	587
SPRING_MAIL_USERNAME	(blank)	your Brevo SMTP login
SPRING_MAIL_PASSWORD	(blank)	your Brevo SMTP key
SPRING_MAIL_PROPERTIES_MAIL_SMTP_AUTH	false	true
SPRING_MAIL_PROPERTIES_MAIL_SMTP_STARTTLS_ENABLE	false	true
NOTIFICATION_FROM_ADDRESS	noreply@finance.local	noreply@your-domain.com
ORIGIN_CERT_CN	unset (defaults localhost)	your-domain.com
APP_MARKET_INIT_ENABLED	true	false (demo seed is loaded instead)

The compose overlay also changes: `docker-compose.yml` + `docker-compose.prod.yml` on VDS (drops the Mailpit dev catcher); `docker-compose.yml` + `docker-compose.demo.yml` locally (adds the demo seed and `demouser`).

Choosing demo vs empty database

Two ways to bring the stack up — same code, different startup seed:

- **Demo** (`make demo`) — `docker-compose.demo.yml` adds the `db-seed` one-shot container that loads `seed/market_data.sql.gz` (~4 years of candles) plus `seed/demo_portfolio.sql` (the sample portfolio), and the `realm-demo.json` realm provisions `demouser` / `Demo123!` who owns that portfolio. Best for clone-and-run showcase. No EVDS/CoinGecko key needed.
- **Empty** (`make empty`) — boots with empty tables. The backend's `MarketDataInitializer` detects empty tables and pulls multi-year history from the upstreams (FX → stocks → commodities) on first run; this takes ~20–30 min and **requires** `EVDS_API_KEY` because stocks and commodities are converted through FX. Set `APP_MARKET_INIT_ENABLED=false` to skip the cold-start fetch entirely.

`make reset` wipes all volumes if you want to switch modes; `make down` just stops without losing data.

Recording GIFs for the setup steps (Kap on macOS)

The placeholders above expect short looping GIFs saved under `docs/imagesandgifs/`. The easiest tool on macOS is **Kap** (free, open-source): <https://getkap.co/>.

1. Open Kap. A floating capture rectangle appears.
2. Drag the rectangle to frame the area to record (the provider's dashboard panel, GitHub secrets page, Cloudflare DNS table, etc.). Resize the rectangle handles to crop.
3. Click **Record**. Perform the steps (click through the UI).
4. Click the Kap menubar icon → **Stop**.
5. In the editor, **Format dropdown** → **GIF**, trim if needed, click **Save**, choose `docs/imagesandgifs/setup-<topic>.gif` matching the filename printed under the placeholder in this doc (e.g. `setup-cloudflare-dns.gif`).

GIFs are committed alongside the doc; the markdown references resolve once the file exists.